CSED232 Principles of Software Construction

Subtypes and Generics

Kyungmin Bae

Department of Computer Science and Engineering

POSTECH

Spring 2026

Subtypes

B is a Subtype of A

- Every B is A
- Every B satisfies A's specification.
- B's specification is at least as strong as A's specification.
  - Each method in B has a stronger specification than the one in A.
  - The class invariant of B is stronger than the class invariant of A.

Example (1)

```
abstract class Vehicle {
    int speed, limit;
    //@ invariant: speed < limit

    //@ requires: speed > 0
    //@ ensures: speed < \old(speed)
    void brake();
}
```

```
class Car extends Vehicle {
    int fuel;
    boolean engineOn;
    //@ invariant: speed < limit
    //@          && fuel >= 0

    //@ requires: !engineOn && fuel > 0
    //@ ensures: engineOn
    void start() { ... }

    //@ requires: speed > 0
    //@ ensures: speed < \old(speed)
    void brake() { ...}
}
```

- Car's specification is stronger than Vehicle's

Example (2)

```
class Car extends Vehicle {
    int fuel;
    boolean engineOn;
    //@ invariant: speed < limit
    //@          && fuel >= 0

    //@ requires: !engineOn && fuel > 0
    //@ ensures: engineOn
    void start() { ... }

    //@ requires: speed > 0
    //@ ensures: speed < \old(speed)
    void brake() { ... }
}
```

```
class HybridCar extends Car {
    int charge;
    //@ invariant: charge >= 0

    //@ requires: !engineOn
    //@     && (charge > 0 || fuel > 0)
    //@ ensures: engineOn;
    void start() { ... }

    //@ requires speed > 0
    //@ ensures: speed < \old(speed)
    //@     && charge > \old(charge)
    void brake() { ... }
}
```

- HybridCar's specification is stronger than Car's

Liskov Substitution Principle

- If B is a subtype of A:
  - An instance of B can always be substituted for an instance of A.
  - code written using supertype A operates correctly for subtype B.

- Subtype B can strengthen specification
  - An overriding method must have a stronger (or equal) specification
  - It is fine to add new methods, as long as they preserve invariants

- Subtype B cannot weaken specification
  - Methods from its supertype cannot be removed
  - An overriding method must not have a weaker specification

Contravariance, Invariance, and Covariance

- Method arguments
  - In an overriding method, argument types may be replaced with supertypes of the original argument types (“contravariance”)
  - However, Java does not allow this due to issues such as method overloading conflicts, limiting overrides to the same argument types (“invariance”)
- Method results
  - In an overriding method, return types may be replaced with subtypes of the original return types (“covariance”)
  - An overriding method must not declare new exceptions, but it may replace declared exceptions with subtypes of the original exceptions.

Example

- Suppose we have the following method in the class `Vehicle`

```
Vehicle recommend(Vehicle ref);
```

- Which of the following are possible forms of this method in `Car`?
  - `Vehicle recommend(Car ref);`
    - ❌ bad
  - `Car recommend(Vehicle ref);`
    - ✅ good
  - `Vehicle recommend(Object ref);`
    - ⚠️ OK, but this is Java overloading
  - `Vehicle recommend(Vehicle ref) throws NoVehicleException;`
    - ❌ bad

Subtyping vs. Subclassing

- Substitution (subtype)
  - specification notion
  - behavior of a subtype object is a subset of supertype spec
- Inheritance (subclass)
  - implementation notion
  - to create an inherited class, write only the differences
- Subclassing is orthogonal to subtyping
  - subclassing does not guarantee subtyping
  - subtyping without subclassing is possible (although not support by compiler)

Q: Is Square a Subtype of Rectangle? (1)

```
class Rectangle {
    int h, w; //@ invariant: h > 0 && w > 0

    Rectangle(int h, int w) {
        this.h = h;
        this.w = w;
    }

    //@ requires: factor > 0
    //@ ensures: w and h are multiplied by factor
    void scale(int factor) {
        w = w * factor;
        h = h * factor;
    }

    //@ requires: neww > 0
    //@ ensures: w is set to neww
    void setWidth(int neww) {
        w = neww;
    }
}
```

```
class Square extends Rectangle {
    //@ invariant: h > 0 && w > 0
    //@          && h == w

    Square(int w) {
        super(w, w);
    }
...
}
```

Q: Is Square a Subtype of Rectangle? (2)

- Which is the best specification option for `Square`'s `setWidth`?

```
//@ requires: h == neww
//@ ensures: w is set to neww
void setWidth(int neww);

//@ requires: neww > 0
//@ ensures: h and w are set to neww
void setWidth(int neww);

//@ requires: neww > 0
//@ ensures: w is set to neww if h = neww;
             throws an exception if h != neww;
void setWidth(int neww);
```

- In all of these cases, `Square` is not a subtype of `Rectangle`!

Mutable Square and Rectangle Unrelated

- `Square` is not a subtype of `Rectangle`
  - Rectangles should allow width and height to change independently
  - Squares violate this expectation

- `Rectangle` is not a subtype of `Square`
  - Squares require equal width and height
  - Rectangles violate this constraint

- Typical solutions
  1. Make both classes immutable
  2. Define both as subclasses of a common (abstract) class/interface

Q: is MutableRect a Subtype of Rect?

```
class Rect {
    int h, w; //@ invariant: h > 0 && w > 0

    Rect(int h, int w) {
        this.h = h;
        this.w = w;
    }

    int getArea() {
        return h * w ;
    }

}
```

```
class MutableRect extends Rect {
    //@ invariant: h > 0 && w > 0

    MutableRect(int h, int w) {
        super(h, w);
    }

    void scale(int factor) {
        w = w * factor;
        h = h * factor;
    }
}
```

Mutable Extensions May Break Subtyping

```
class AreaCache {
    //@ invariant: rect.getArea() == cached
    final Rect rect;
    final int cached;

    AreaCache(Rect rect) {
        this.rect = rect;
        this.cached = rect.getArea();
    }
    ...
}
```

```
class Client {

    void run() {
        Rect r = new MutableRect(2, 3);
        AreaCache cache = new AreaCache(r);

        // Invariant violated!
        ((MutableRect) r).scale(10);
    }

}
```

- `AreaCache` expects that `Rect` is immutable
- `Client` invalidates the invariant of `AreaCache`!

- But new fields added to the subtype may be safely modified

Benefits of Immutability

- No worries about representation exposure
- No worries about key mutation errors
- Subtyping usually works the way you expect

Generics

Motivating Example

- A Box class that operates on objects of any type

```
public class Box {
    private Object object;

    public void set(Object obj) { object = obj; }

    public Object get() { return object; }
}
```

- No way to verify how the class is used at compile time!

```
Box box = new Box();
box.set(Integer.valueOf(3));


String str = (String) box.get();    // no error at compile-time
```

A Generic Version of Box

- A Box class with a type parameter

```
public class Box<T> {
    private T t;

    public void set(T t) { this.t = t; }

    public T get() { return t; }
}
```

- Stronger type checks at compile time

```
Box<Integer> box = new Box<Integer>();
box.set(Integer.valueOf(3));

String str = (String) box.get();    // compile-time error occurs
```

Java Generics

- Declarations parameterized by types
  - When defining classes, interfaces, and methods
  - Reuse the same code with different types

- Benefits
  - Stronger type checks at compile time
  - Elimination of casts
  - Enabling implementation of generic algorithms

Declaring and Instantiating Generics

- Declaration

```
class Name<TypeVar1, ..., TypeVarN> { ... }
interface Name<TypeVar1, ..., TypeVarN> { ... }
```

- Convention for type variables
  - Use single-letter names, e.g., T for Type, E for Element, K for Key, …

- To instantiate a generic class/interface, client supplies type arguments

```
Name<Type1, ..., TypeN>
```

Example (1)

```
public class OrderedPair<K, V> implements Pair<K, V> {

    private K key;
    private V value;

    public OrderedPair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() {
        return key;
    }

    public V getValue() {
        return value;
    }
}
```

Example (2)

- Instantiating with explicit types

```
Pair<String, Integer> p1 = new OrderedPair<String, Integer>("Even", 8);
Pair<String, String> p2 = new OrderedPair<String, String>("hello", "world");
```

- Inferring types from declaration

```
OrderedPair<String, Integer> p1 = new OrderedPair<>("Even", 8);
OrderedPair<String, String> p2 = new OrderedPair<>("hello", "world");
```

- Parameterized type arguments

```
OrderedPair<String, Box<Integer>> p = new OrderedPair<>("primes", new Box<>());
```

Not All Generics are for Collections

```
public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public void setKey(K key) {
        this.key = key;
    }
    public void setValue(V value) {
        this.value = value;
    }
    public K getKey() { return key; }
    public V getValue() { return value; }
}
```

```
public class Util<K, V> {

    public boolean compare(Pair<K, V> p1,
                           Pair<K, V> p2) {
        return p1.getKey().equals(
                    p2.getKey()) &&
                        p1.getValue().equals(
                            p2.getValue());
    }
}
```

- Weakness
  - Must create a new Util<K, V> instance for each different K and V
  - even though the method has no instance-specific behavior.

Generic Methods

- The following is a revised version of `Util`

```
public class Util {

    public static <K, V> boolean compare(Pair<K, V> p1, Pair<K, V> p2) {
        return p1.getKey().equals(p2.getKey()) && p1.getValue().equals(p2.getValue());
    }

}
```

  - Extra type parameters are declared in the method (not at the class level)

Example: Invoking Generic Methods

- With explicit type arguments

```
Pair<Integer, String> p1 = new Pair<>(1, "apple");
Pair<Integer, String> p2 = new Pair<>(2, "pear");

boolean same = Util.<Integer, String>compare(p1, p2);
```

- With type inference

```
Pair<Integer, String> p1 = new Pair<>(1, "apple");
Pair<Integer, String> p2 = new Pair<>(2, "pear");

boolean same = Util.compare(p1, p2);
```

Bounded Type Parameters

- Restrict the types that can be used as type arguments

```
<T extends B>
```

- Type parameter can have multiple bounds

```
<T extends ClassT & InterfaceT1>
<T extends ClassT & InterfaceT1 & InterfaceT2>
…
```

Example (1)

```
public class Box<T> {
    private T t;

    public void set(T t) { this.t = t; }

    public T get() { return t; }

    public <U extends Number> void inspect(U u) {
        System.out.println("T: " + t.getClass().getName());
        System.out.println("U: " + u.getClass().getName());
    }

    public static void main(String[] args) {
        Box<Integer> integerBox = new Box<Integer>();
        integerBox.set(Integer.valueOf(10));
        integerBox.inspect("some text");        // error: String does not extend Number!
    }
}
```

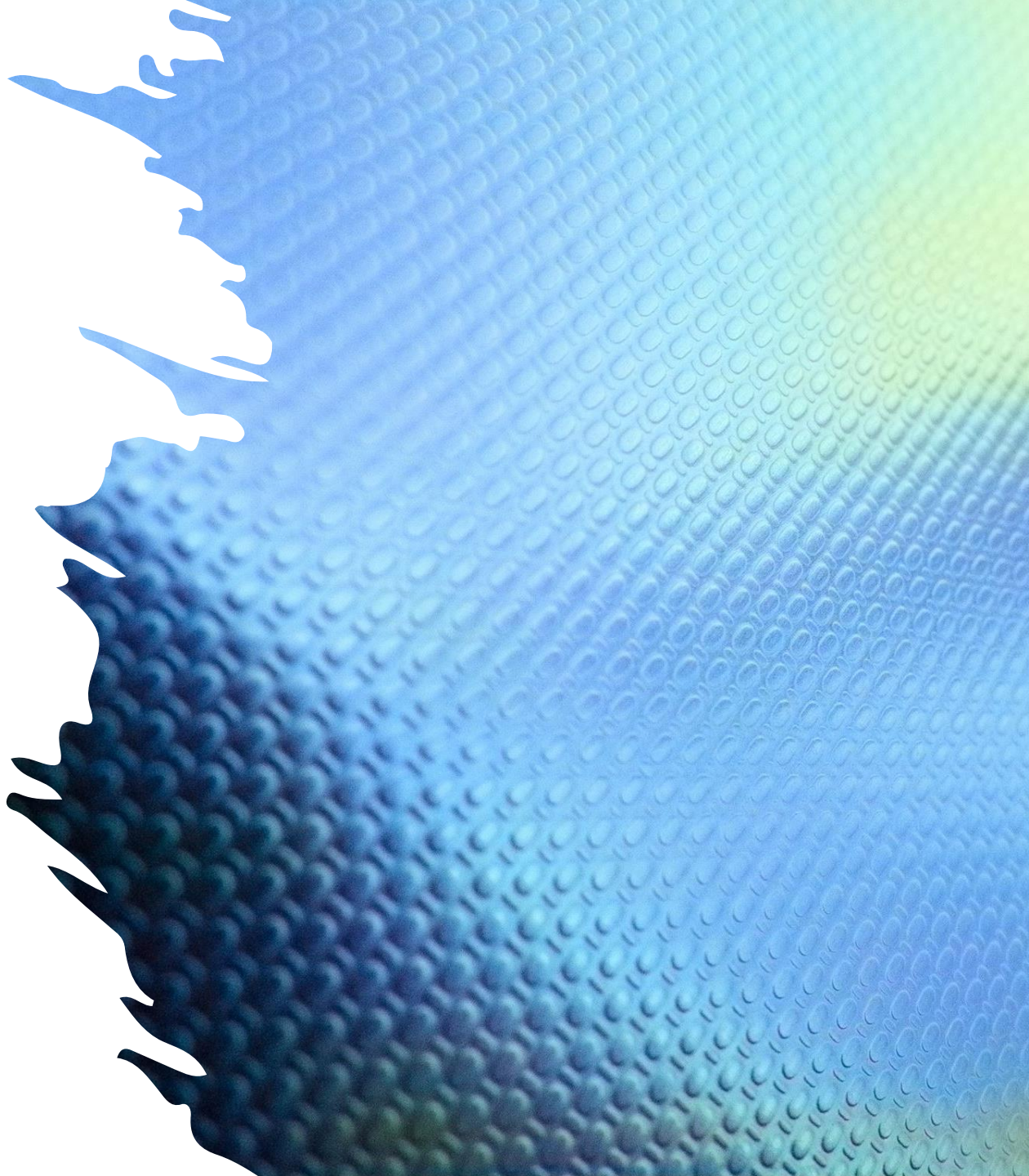
Example (2)

- Code can perform any operation permitted by the bound

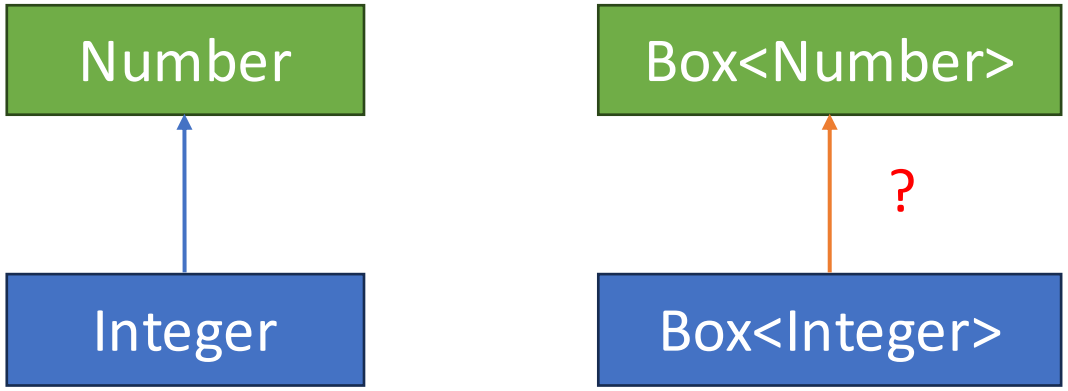
```
public class NaturalNumber<T extends Integer> {
    private T n;

    public boolean isEven() {
        return n.intValue() % 2 == 0;  // invokes intValue defined in Integer
    }

    ...
}
```

```
public static <T extends Comparable<T>> int countGreaterThan(T[] anArray, T elem) {
    int count = 0;
    for (T e : anArray)
        if (e.compareTo(elem) > 0)    // invokes compareTo defined in Comparable
            ++count;
    return count;
}
```

Generics and Subtypes



Generics and Subtyping



- `Integer` is a subtype of `Number`
- Is `Box<Integer>` a subtype of `Box<Number>`?

- Comparing specifications to find out …

Box<Number> and Box<Integer>

```
public class Box<Number> {
    private Number t;

    public void set(Number t) {
        this.t = t;
    }

    public Number get() { return t; }
}
```

```
public class Box<Integer> {
    private Integer t;

    public void set(Integer t) {
        this.t = t;
    }

    public Integer get() { return t; }
}
```

- For set, which specification is stronger?
  - Box<Number>.set > Box<Integer>.set (contravariance)
- For get, which specification is stronger?
  - Box<Number>.get < Box<Integer>.get (covariance)

- Box<Number> and Box<Integer> are not subtypes of each other!

Question

```
interface Set<E> {
    // Adds all elements in the collection c to the set
    void addAll(______ c);
}
```

- What is the best type for addAll's parameter?
  - `void addAll(Set<E> c)`
    - ❌ Too restrictive: does not allow clients to pass other collections, such as List<E>
  - `void addAll(Collection<E> c)`
    - ❌ Still too restrictive: client cannot pass a List<Integer> to addAll for a Set<Number>
  - `<T extends E> void addAll(Collection<T> c)`
    - ✅ More flexible: now client can pass List<Integer> to addAll for Set<Number>
- The implementation will not know what the element type T is.
  - a more concise syntax would be helpful.

Wildcard

- Represents an unknown types

```
?
```

- Upper bounded

```
? extends T
```

- Lower bounded

```
? super T
```

- `<? extends T>` denotes an unspecified subtype of `T`
- `<?>` is a shorthand for `<? extends Object>`
- `<? super T>` denotes an unspecified supertype of `T`

Examples (1)

```
interface Set<E> {
    void addAll(Collection<? extends E> c);
}
```

- More flexible than
  - `void addAll(Collection<E> c)`

- More concise (but equally powerful) than
  - `<T extends E> void addAll(Collection<T> c)`

Examples (2)

```
public static double sumOfList(List<? extends Number> list) {
    double s = 0.0;
    for (Number n : list) s += n.doubleValue();
    return s;
}
```

- Each element of list can be assigned to `Number`
  - because it is a subtype of `Number`

- Notice that `List<? super Number> list` would not work!
  - the element type of `list` could be a strict supertype of `Number`

Examples (3)

```
public static void addNumbers(List<? super Integer> list) {
    for (int i = 1; i <= 10; i++) {
        list.add(i);
    }
}
```

- Integer `i` can be added to list
  - because each element of list is a supertype of Integer

- What happens if `List<? extends Number> list` is used?
  - A compile error, as the element type could be a strict subtype of `Integer`!

PECS: Producer Extends, Consumer Super

- Use `<? extends T>`
  - for input variables that get values (from a producer)
- Use `<? super T>`
  - for output variables that put values (into a consumer)
- Use neither (i.e., T)
  - for variables used as both input and output

Example

- copyTo for Box

```
static <T> void copyTo(Box<? extends T> src, Box<? super T> dst) {
    dst.set(src.get());
}
```

- Callers get the subtyping they want

```
copyTo(numBox, numBox);    // OK
copyTo(intBox, numBox);    // OK
copyTo(numBox, intBox);    // compile time error
```

? Versus Object

- ? Indicates a specific but unknown type

- `Box<?>` vs. `Box<Object>`
  - Valid: `Box<?> box = new Box<String>();`
  - Invalid: `Box<Object> box = new Box<String>();`

- `Box<Foo>` vs. `Box<? extends Foo>`
  - `Box<? extends Foo>`: element type is a specific unknown subtype of Foo
    - E.g.: `Box<? extends Animal>` can store only a Dog, not a Cat, if the actual type is Dog
  - `Box<Foo>`: can store any subtype of Foo
    - E.g.: `Box<Animal>` can store both a Dog and a Cat

Example: Which are Legal?

```
Box<? extends Number> box;
Object obj;
Number num;
Integer int;

box = new Box<Object>();     // error
box = new Box<Number>();
box = new Box<Integer>();

box.set(obj);     // error
box.set(num);     // error
box.set(int);     // error
box.set(null);


obj = box.get();
num = box.get();
int = box.get();     // error
```

```
Box<? super Number> box;
Object obj;
Number num;
Integer int;

box = new Box<Object>();
box = new Box<Number>();
box = new Box<Integer>();     // error

box.set(obj);     // error
box.set(num);
box.set(int);
box.set(null);


obj = box.get();
num = box.get();     // error
int = box.get();     // error
```

Wildcards and Subtyping

- Box<?> is a common parent of Box<Number> and Box<Integer>

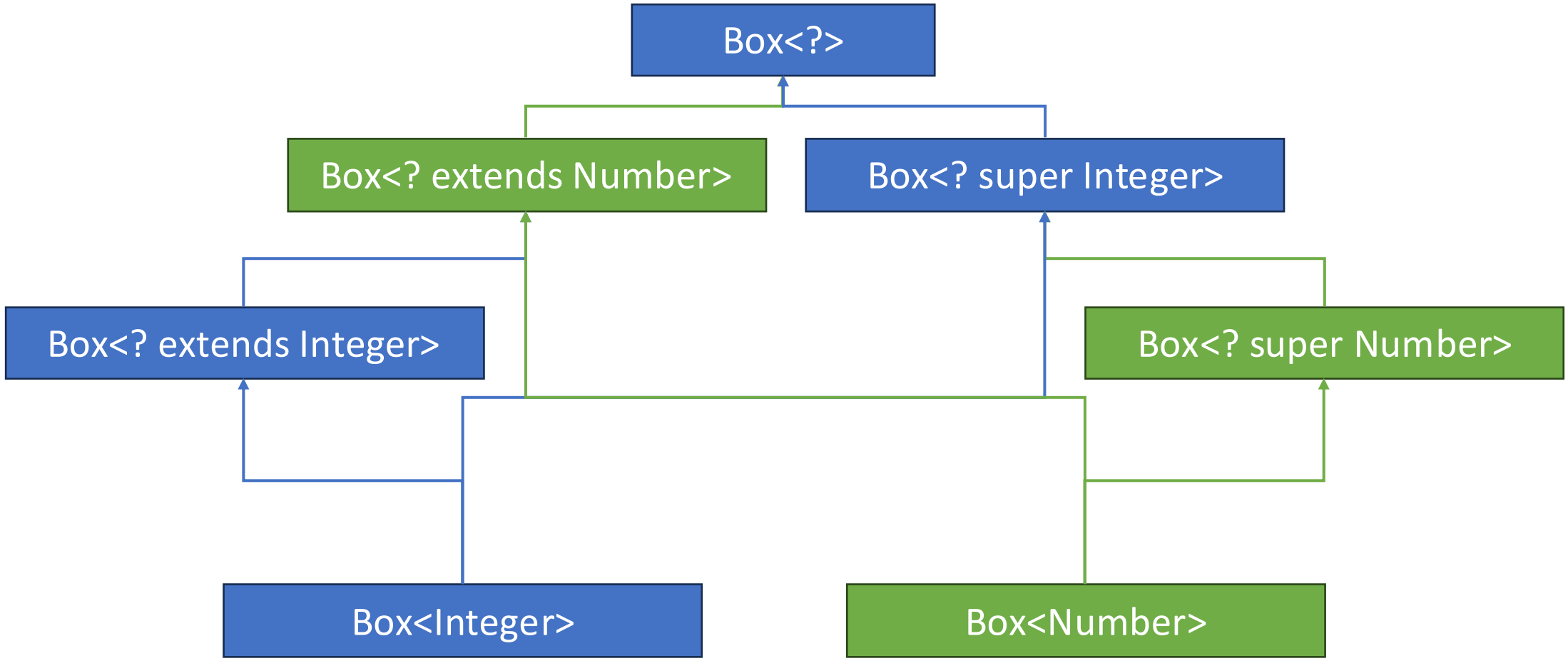
```
Box<Integer> intBox = new Box<>();
Box<Number> numBox  = intBox;          // compile-time error
Box<?> anyBox = intBox;                // OK
```

- Box<? extends Integer> is a subtype of Box<? extends Number>

```
Box<? extends Integer> intBox = new Box<>();
Box<? extends Number>  numBox = intList;       // OK.
```

- Box<? super Number> is a subtype of Box<? super Integer>

Wildcards and Subtyping



References

- Core Java
  - Chapter 8
- Effective Java
  - Chapter 5
- “Program Development in Java” by B. Liskov
  - Chapter 7
- Wikipedia
  - Liskov substitution principle

Question?

More on Java Subtypes and Generics

Java Arrays and Subtype (1)

- Java arrays
  - `Type[]`: An array holding elements of type `Type`
  - Get an element: `x[i]`
  - Set an element: `x[i] = e`

- If T1 is a subtype of T2, how should T1[] and T2[] be related?
  - In theory, T1[] and T2[] should be unrelated.
  - In Java, T1[] is considered a subtype of T2[] (covariance), which is not true subtyping!
  - The main reason for this design is backward compatibility.

Java Arrays and Subtype (2)

- What can happen:

```
1  String[] strs = new String[10];  // Creates an array of Strings
2  Object[] objs = strs;            // Allowed due to Java's array covariance
3
4  objs[1] = Integer.valueOf(1);    // Compiles, but inserts an Integer into a String array
5  int len = strs[1].length();      // strs[1] == objs[1], so this fails at runtime!
```

  - Not type safe!

- In Java, this is checked "dynamically"
  - Each array maintains its actual run-time type
  - Storing an incompatible type causes `ArrayStoreException` (e.g., line 4)

Type Erasure

- All generic types become Object (or bound types) at compile time
  - Replace all type parameters in generic types with their bounds
  - Insert type casts if necessary to preserve type safety

- At run-time, all generic instantiations have the same type

```
Box<String> box1 = new Box<>();
Box<Integer> box2 = new Box<>();
System.out.println(box1.getClass() == box2.getClass());      // true!
```

Consequence of Type Erasure (1)

- Cannot use `instanceof` with parameterized types

```
public static <E> void rtti(List<E> list) {
    if (list instanceof ArrayList<Integer>)  // compile-time error
    // ...

    if (list instanceof ArrayList<?>)        // OK
    // ...
}
```

Consequence of Type Erasure (2)

- Cannot cast to a parameterized type, except for unbounded wildcards

```
Box<Integer> box1 = new Box<>();
Box<Number>  box2 = (Box<Number>) box1;  // compile-time error
Box<?> box3 = box1;                      // OK
```

- The following gives an unchecked warning

```
Box<Number> box4 = (Box<Number>) box3;   // Compiles, but unsafe
```

  - A ClassCastException may occur at runtime (why?)

Consequence of Type Erasure (3)

- Cannot directly create an instance of a parameterized type

```
public class SomeClass<T> {
    private T t;

    public SomeClass() {
        this.t = new T(); // compile-time error
    }
}
```

Creating Instance of Parameterized Types

- Do not instantiate parameterized types unless necessary
  - In many cases, client can provide instances

- There are several ways to create parameterized types indirectly
  - E.g., using a ``factory'' method

```
public class SomeClass<T> {
    ...
    public SomeClass(Supplier<T> supp) {
        this.t = supp.get();
    }
}
```

```
public interface Supplier<T> {
    T get();
}
```

  - A subclass of `Supplier<T>` defines how to create an instance

Consequence of Type Erasure (4)

- Cannot create arrays of parameterized types (even when instantiated)

```
Object[] strLists = new List<String>[2];    // compile-time error
```

- Otherwise, due to type erasure, runtime type check would fail.

```
strLists[0] = new ArrayList<String>();   // OK
strLists[1] = new ArrayList<Integer>();  // cannot throw ArrayStoreException
```

- Use generic collections instead in such cases.

```
List<List<String>> list = new ArrayList<>();
list.add(new ArrayList<String>());
```